

What is CRI (Container Runtime Interface)?

What is CRI?

Simple Definition

CRI (Container Runtime Interface) is a standard interface that allows Kubernetes to communicate with different container runtimes.

Beginner Definition

CRI is a bridge (or translator) between Kubernetes and the Container Runtime.

Easy Real-Life Example 🏠

Imagine a school:

- 🧑 Principal = Kubernetes
- 🗣️ Translator = CRI
- 🧑 Worker = Container Runtime

The principal gives instructions.

The translator converts the instructions into a language the worker understands.

Similarly,

Kubernetes

|



CRI

|



Container Runtime

(containerd / CRI-O)

|



Containers

Memory Trick:

CRI = Translator between Kubernetes and Container Runtime.

Why was CRI Introduced?

Before CRI:

- Kubernetes worked mainly with Docker.
- It was tightly coupled with Docker.
- Supporting other container runtimes was difficult.

To solve this problem, Kubernetes introduced **CRI**, allowing it to work with multiple runtimes.

Brief History of CRI

Before 2016

- Docker became the most popular container platform.
 - Kubernetes used **Docker** to run containers.
-

Kubernetes 1.5 (2016)

- Kubernetes introduced **CRI (Container Runtime Interface)**.
 - Kubernetes could now support multiple container runtimes.
-

After CRI

Kubernetes started supporting runtimes like:

-  containerd
-  CRI-O
-  Docker (through **Dockershim**)

Kubernetes 1.24 (2022)

- **Dockershim was removed.**
- Kubernetes no longer communicates directly with Docker.
- It now uses **CRI-compatible runtimes** like:
 - containerd
 - CRI-O

Note: Docker is still useful for **building container images**, but Kubernetes uses a CRI-compatible runtime to run those containers.

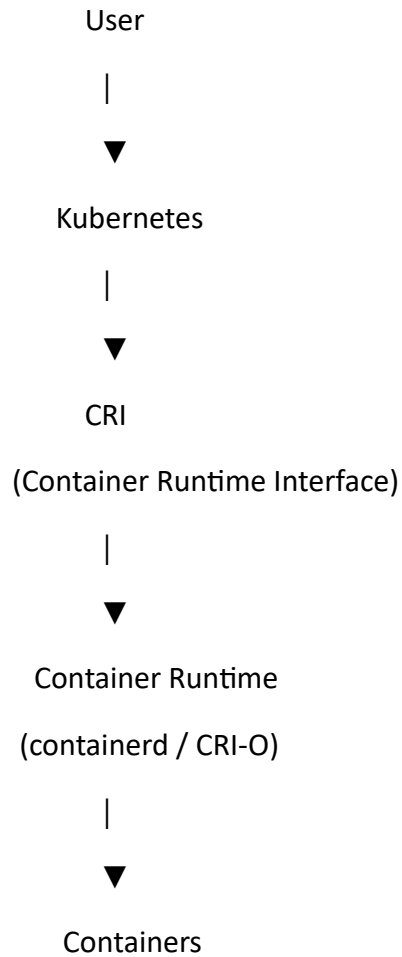
Popular Container Runtimes

Container Runtime	Description
containerd	Most commonly used runtime with Kubernetes.
CRI-O	Lightweight runtime built specifically for Kubernetes.
Docker	Popular tool for building container images. (Not used directly as the runtime in modern Kubernetes.)

Advantages of CRI

- Supports multiple container runtimes.
 - Reduces Kubernetes' dependency on Docker.
 - Makes Kubernetes more flexible.
 - Easier to maintain and upgrade.
 - Uses a standard communication interface.
-

Easy Diagram

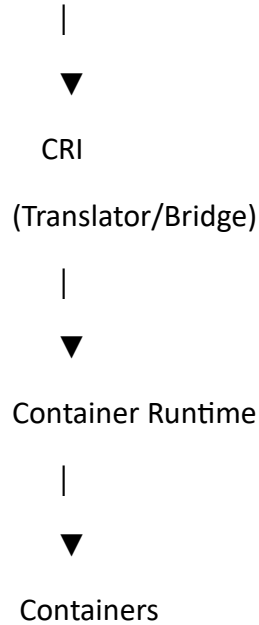


Quick Revision

- **CRI = Container Runtime Interface**
- Acts as a **bridge** between Kubernetes and the Container Runtime.
- Introduced in **Kubernetes 1.5 (2016)**.
- **Dockershim was removed in Kubernetes 1.24 (2022)**.
- Modern Kubernetes commonly uses **containerd** and **CRI-O**.
- CRI allows Kubernetes to support multiple container runtimes.

Easy Memory Trick ★

Kubernetes



Interview Definition

CRI (Container Runtime Interface) is a standard interface that enables Kubernetes to communicate with different container runtimes such as containerd and CRI-O.

Remember

-  Kubernetes = Manager
-  CRI = Bridge/Translator
-  Container Runtime = Runs Containers
-  Container = Runs the Application

1. What is gRPC?

Simple Definition

gRPC (Google Remote Procedure Call) is a communication protocol that allows two programs to communicate with each other quickly and efficiently.

Beginner Definition

gRPC is like a messenger that helps Kubernetes talk to the Container Runtime.

Easy Real-Life Example 📞

Imagine:

- 👤 Principal = Kubernetes
- 📞 Telephone = gRPC
- 🧑 Worker = Container Runtime

The principal wants to tell the worker:

- Start a container
- Stop a container
- Restart a container

Instead of walking there, the principal uses the **telephone (gRPC)**.

Kubernetes

|

| gRPC (Communication)



Container Runtime

|



Container

Why is gRPC Used?

- Fast communication
- Secure communication
- Efficient communication
- Works across different programming languages

In Kubernetes

Kubernetes uses **gRPC** to communicate with the **CRI (Container Runtime Interface)**.

Kubernetes

|



CRI

|

| gRPC



Container Runtime

Memory Trick



gRPC = Telephone between Kubernetes and Container Runtime

2. What is Dockershim?

Simple Definition

Dockershim was a compatibility layer that allowed Kubernetes to communicate with Docker.

Beginner Definition

Dockershim was a bridge between Kubernetes and Docker.

Why was Dockershim Needed?

Earlier,

Docker **did not support CRI directly.**

But Kubernetes wanted to communicate using **CRI**.

So Kubernetes used **Dockershim** as a translator.

Kubernetes

|



Dockershim

|



Docker Engine

|



Container

Real-Life Example 🏠

Imagine:

- 👤 Principal speaks English.
- 🙋 Worker speaks Urdu.

They can't understand each other.

A translator is needed.

That translator is **Dockershim**.

What Happened to Dockershim?

Before Kubernetes 1.24

Kubernetes

|



Dockershim

|



Docker

Kubernetes communicated with Docker using Dockershim.

After Kubernetes 1.24

Dockershim was removed.

Now Kubernetes communicates directly with CRI-compatible runtimes.

Kubernetes

|



CRI

|



containerd

or

Kubernetes

|



CRI

|



CRI-O

Why Was Dockershim Removed?

- Docker was not built as a native CRI runtime.
 - Maintaining Dockershim added extra complexity.
 - Kubernetes wanted a simpler architecture.
 - **containerd** and **CRI-O** already support CRI directly.
-

Docker vs Dockershim

Docker

Creates and manages containers

Container platform

Still used to build container images

Dockershim

Bridge between Kubernetes and Docker

Compatibility layer

Removed in Kubernetes 1.24

gRPC vs Dockershim

gRPC

Communication protocol

Helps software communicate

Still used in Kubernetes

Fast and efficient

Dockershim

Compatibility layer

Helped Kubernetes communicate with Docker

Removed in Kubernetes 1.24

No longer used

Complete Flow (Old Architecture)

User

|



Kubernetes

|



Dockershim

|



Docker

|



Container

Complete Flow (Modern Architecture)

User



Kubernetes



CRI



gRPC



containerd / CRI-O



Container

Quick Revision

gRPC

- Stands for **Google Remote Procedure Call**.
- Communication protocol.
- Used by Kubernetes to communicate with the Container Runtime through CRI.
- Fast and efficient.

Dockershim

- Compatibility layer between Kubernetes and Docker.
 - Allowed Kubernetes to use Docker before CRI support.
 - Removed in **Kubernetes 1.24**.
 - Replaced by direct use of **CRI-compatible runtimes** like **containerd** and **CRI-O**.
-

Easy Memory Trick

-  gRPC = Telephone (Communication)
-  Dockershim = Bridge (Kubernetes ↔ Docker)
-  CRI = Standard Interface
-  containerd / CRI-O = Container Runtimes
-  Kubernetes = Manager of Containers

One-Line Formula

Kubernetes → CRI → gRPC → Container Runtime → Container

This is the **modern Kubernetes architecture** used today.

CRI, gRPC & DOCKERSHIM – ALL IN ONE (EASY WAY)

1) What is CRI?

- CRI (Container Runtime Interface) is a standard interface that allows Kubernetes to talk to different Container Runtimes.
- It acts as a bridge between Kubernetes and the Container Runtime.

2) What is gRPC?

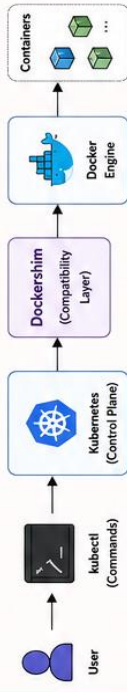
- gRPC (Google Remote Procedure Call) is a high performance, open source communication protocol.
- Kubernetes uses gRPC to communicate with the Container Runtime through CRI.

3) What was Dockershim?

- Dockershim was a compatibility layer that allowed Kubernetes to communicate with Docker.
- Docker did not support CRI directly, so Dockershim acted as a bridge.
- It was removed in Kubernetes v1.24.

BEFORE KUBERNETES 1.24 (Old Architecture)

Kubernetes used Dockershim to talk to Docker

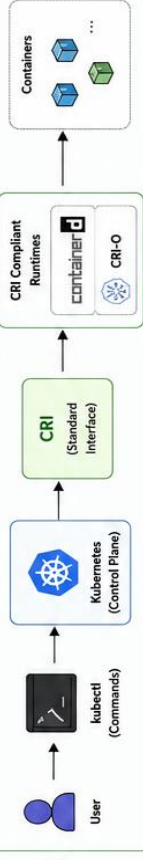


How it worked?

1. User sends command using kubectl
2. Kubernetes needs to run a container
3. It sends request to Dockershim
4. Dockershim translates the request for Docker
5. Docker Engine runs the container

AFTER KUBERNETES 1.24 (New Architecture)

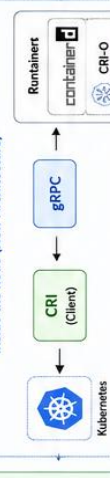
Dockershim is removed. Kubernetes talks to CRI compliant runtimes directly



How it works now?

1. User sends command using kubectl
2. Kubernetes talks to CRI (standard interface)
3. CRI uses gRPC to communicate with the runtime
4. Container Runtime runs the container

Inside the node (New Architecture)



KEY POINTS TO REMEMBER

- ★ CRI = Standard interface between Kubernetes and Container Runtimes.
- ★ gRPC = Communication protocol used by CRI to talk to the runtime.
- ★ Dockershim = Old bridge to connect Kubernetes with Docker.
- ★ Removed in Kubernetes v1.24.
- ★ Modern Kubernetes uses containerd or CRI-O (CRI compliant runtimes).

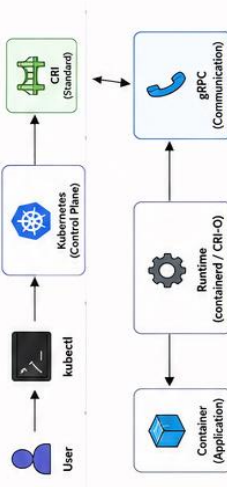
EASY MEMORY TRICK



COMPARISON TABLE

Feature	CRI	gRPC	Dockershim
Type	Interface (Standard)	Protocol (Communication)	Compatibility Layer (Bridge)
Purpose	Standard communication between K8s and Runtime	High performance communication	Allowed K8s to use Docker
Used For	All CRI compliant runtimes	Used by CRI to talk to runtime	Only for Docker
Status	Still used (Mandatory)	Still used (Mandatory)	Removed in K8s v1.24
Examples	containerd, CRI-O	N/A	Docker (older)

COMPLETE MODERN FLOW (Easy View)



ONE LINE SUMMARY:

CRI is the standard bridge, gRPC is the language/telephone, and Dockershim was the old bridge for Docker (now removed).